FUND-CRITICAL SMART-CONTRACT REVIEW

Generic.Money (GUSD)

Bespoke ERC-7575 omnichain stablecoin — core protocol & cross-chain bridging

VERDICT

CLEAN core • RESIDUAL dependencies

No push-button, fund-critical vulnerability identified across ~6,900 lines of fully bespoke Solidity. Notably well-engineered and defensively coded despite being unaudited.

```
TARGET      generic-money/generic-protocol (core) + MetaFi-Labs/generic-bridging (omnichain)
SCOPE       Vault accounting · pricing/peg · oracle · mint authority · yield/rewards · cross-chain mint · predeposit
METHOD      Line-by-line static analysis + pure-arithmetic invariant validation (no runnable PoC — see note)
TVL         ~$1.2M · selected for HIGHEST P(bug): fully bespoke, novel design, no external audit
DATE        2026-06-13
```

1 • Why this target

Selected by **bug-likelihood, not payout**. Generic.Money is ~4,838 lines of fully bespoke core (not a fork) implementing a novel **ERC-7575 multi-asset vault** with hybrid synchronous-deposit / dynamic-redemption pricing, a peg-defense mechanism, and omnichain mint+bridge — plus a separate ~2,075-line cross-chain bridging stack. It carries **no external audit** (only self-authored security notes). Novel + unaudited + custom decimal/cross-chain math is exactly the profile most likely to hide a fund-critical bug.

2 • Verdict at a glance

SURFACE	FINDING	STATUS
Decimal conversion & rounding	All rounding favors protocol (mint Ceil, redeem Floor); 1:1 share model resists inflation	CLEAN
Pricing / peg defense	min/max oracle pricing robustly blocks depeg arbitrage (verified algebraically)	CLEAN
Atomic round-trip (flash)	Secondary-action fee + transient tx flags penalize same-tx deposit+withdraw	CLEAN
Oracle (PriceFeedManager)	Dynamic decimals normalization + staleness + positivity checks	CLEAN
Mint authority	onlyOwner mint; GUSD wraps GenericUnit 1:1 (lock-and-mint)	CLEAN
Yield / rewards extraction	Role-gated; cannot touch vault collateral; only surplus buffer minted	CLEAN
Cross-chain mint (bridge)	Dual adapter auth + LZ peer/endpoint + exactly-once + rollback	CLEAN
Predeposit lifecycle	Mutually-exclusive state machine; entries deleted on consume	CLEAN

3 • Where the bugs usually hide — and what we found

3.1 Decimal conversion & share inflation (the team's own #1 worry)

Assets are normalized 6↔18 decimals on every flow. Every rounding decision is **protocol-favoring**: mint cost rounds `Ceil`, redeem payout rounds `Floor`, deposit shares round `Floor`. Crucially, shares are minted **1:1 with normalized assets** — there is no ERC-4626 proportional share price to inflate, neutralizing the classic first-depositor / donation inflation attack. `mint()` enforces non-zero asset cost (Ceil), preventing free-share extraction at dust amounts.

3.2 Peg / depeg arbitrage

Deposits credit the incoming asset at `min(oracle, $1)` with a fixed $1 share price; redemptions charge the outgoing asset at `max(oracle, $1)` with a dynamic share price `min($1, backing/supply)`. We verified algebraically that extracting either the cheap or the expensive collateral asset is, at best, break-even — the min/max bracketing structurally removes depeg arbitrage profit. Same-asset round-trips inside one transaction are penalized by a secondary-action fee, with the in-tx deposit/withdraw flags held in native `transient` storage (auto-reset per tx).

3.3 Oracle handling — the Vena lesson, done right

Unlike a prior target where a hardcoded 8-decimal assumption created a residual, **PriceFeedManager reads each feed's `decimals()` dynamically** and normalizes to 18 decimals, alongside positive-answer and staleness (heartbeat + buffer) checks. No oracle decimal/staleness bug.

3.4 Cross-chain mint authority (highest-value surface)

On L2, inbound bridge messages **mint** GenericUnit — the most dangerous lever in the system. Authentication is layered and sound: the LayerZero endpoint → OApp peer check → coordinator requires `msg.sender` to be a registered local adapter and `remoteSender` to be the registered remote adapter. Each L2 mint is backed by a matching L1 lock/escrow, with the L1 coordinator sanity-checking its own escrow balance delta. Replay is covered by LayerZero exactly-once delivery; failed settlements are caught (not reverted) and recoverable only via `rollback` — so there is no retry-then-rollback double-spend, and a failed message never mints.

3.5 Predeposit lifecycle

The predeposit state machine (`DISABLED → ENABLED → {DISPATCHED xor WITHDRAWN}`) is mutually exclusive: both terminal transitions require the `ENABLED` state and there is no path between DISPATCHED and WITHDRAWN. Each per-(owner, recipient) entry is **deleted before release** on bridge or withdraw, so a predeposit can be bridged or refunded — never both.

4 • Residual / watch items (not vulnerabilities)

R1 • External strategy trust. Backing value — and therefore the redemption price — depends on external ERC-4626 strategies (Morpho / Aave / Sky) via `convertToAssets()`. A lossy or manipulable strategy would directly impair redemption. Donation-inflation is not profitable here (the benefit accrues pro-rata to all holders and exceeds attacker cost), and the named strategies resist it — but this is the protocol's main external dependency.

R2 • Privileged roles. `PRICE_FEED_MANAGER` can set any price feed (a powerful misprice lever); rewards/yield/predeposit/emergency managers and admin govern their domains. Standard centralization trust, not a code defect.

R3 • No coordinator-level replay guard. The bridge relies entirely on the underlying canonical bridges (LayerZero / Linea) for exactly-once delivery. Registering a future non-canonical adapter without its own deduplication would reintroduce replay-mint risk. Architectural note for governance.

5 • Method & honesty note

This review was performed by exhaustive line-by-line static analysis and independent pure-arithmetic validation of the core invariants (decimal scaling, deposit/redeem round-trips, peg pricing brackets). The Foundry toolchain could not be installed in this environment (release-binary downloads were blocked), so **no executable proof-of-concept was run**. The "CLEAN" verdict is therefore a reasoned proof-of-clean over the reviewed surfaces, not a fabricated finding — consistent with the discipline of never claiming a bug without a working exploit. A follow-up with a live Foundry fork harness is the recommended next step to convert this into test-backed assurance, focused on R1 (strategy-valuation interaction) and the bridging stack under a non-canonical adapter.

Prepared by Viktor • Fund-Critical Bug-Hunt methodology • Confidential. Findings reflect the repository state reviewed on 2026-06-13 and the listed scope only.